

40. RBAC Policy DSL Compiler with Static Security & Privilege Escalation Detection

Course Name: Compiler Design

Course Code: CS1202

Name: Pinikeshi Meghana

Roll No: 24CSB0A54

Sec: CSE-A

Document Details: Week 6 Deliverables

Week 6 - RBAC DSL Grammar Design

In Week 6, the focus was on implementing the lexical analyzer (lexer) and the parser for the Role-Based Access Control (RBAC) Domain Specific Language (DSL) compiler. The main goal was to process RBAC policy files, recognize language tokens, parse them according to the designed grammar, and construct a symbol table representing roles, users, and permissions.

Additionally, an Abstract Syntax Tree (AST) was generated to represent the hierarchical structure of the policy, and syntax error detection was integrated to handle invalid policy definitions. This week laid the foundation for semantic analysis in the subsequent week.

Objectives:

- Implement a lexer to tokenize RBAC DSL inputs.
- Build a parser that validates the grammar and populates the symbol table.
- Generate an AST representing the relationships among roles, users, and permissions.
- Detect and report syntax errors during parsing.

1. Lexer Implementation

Purpose

The lexer (lexical analyzer) reads the RBAC DSL policy file and breaks it into tokens, which are the smallest meaningful units in the language. This is the first phase of the compiler.

Key Token Types

Token	Description
ROLE	Represents a role declaration (role)
USER	Represents a user declaration (user)
IDENTIFIER	Names of roles, users, or other identifiers
EXTENDS	Keyword for role inheritance
LBRACE / RBRACE	{ and } for block delimiters
EQUALS	= sign for assignments
PERMISSIONS	Keyword for permissions assignment
ROLES	Keyword for roles assignment to users

LBRACKET / RBRACKET	[and] for lists
COMMA	, to separate list elements
EOF	End of file token

How It Works

1. Reads the DSL file line by line.
2. Skips empty lines and comments.
3. Adds spaces around symbols ({}, =, [], ,) for easier splitting.
4. Splits each line into words/parts.
5. Classifies each part into one of the token types.
6. Appends all tokens to a list, ending with an EOF token

Output / Demonstration

- On running the lexer, it prints the number of tokens generated.
- Optionally, each token can be printed for debugging. (There is a toggle to either print all tokens or to just wrap print the tokens by number of tokens)

2. Parser Implementation & AST Construction

Purpose

The parser takes the tokens generated by the lexer and converts them into a structured format representing the DSL program. This phase checks the syntax and builds the Abstract Syntax Tree (AST), which represents the hierarchical relationships between roles, users, permissions, and inheritance.

Parser Functionality

1. **Token Consumption**
The parser reads tokens one by one and identifies roles, users, permissions, and inheritance based on the grammar.
2. **Block Handling**
Each role or user block is identified by { and }. All assignments inside the block are linked to the current role or user.
3. **Assignments**
 - permissions = [read, write] → assigned to role.

- roles = [Manager, Tester] → assigned to user.

4. Inheritance

- role Manager extends Admin → adds Admin as a parent role.

5. AST Node Creation

- Each role and user becomes a node in the AST.
- AST nodes store:
 - Name
 - Permissions (for roles)
 - Roles (for users)
 - Parent roles (for inheritance)
- Nodes are connected to represent hierarchical relationships.

6. Syntax Error Detection

- Any unexpected tokens or malformed lines generate syntax error messages with clear explanation of where the syntax error is identified

Example:

```
[Syntax Errors]
[SYNTAX ERROR] Role 'Intern' missing '{'
```

Key classes: Role, User, ASTNode

Parser Logic

1. Initialize symbol table and AST node list.
2. Iterate over tokens:
 - When a ROLE token is found → create a Role object and AST node.
 - When a USER token is found → create a User object and AST node.
 - For assignments (permissions or roles), update both the object in the symbol table and its AST node.
3. End of block (}) signals completion of current role or user.
4. Collect syntax errors for any unexpected token or line.

Output / Demonstration

- Prints symbol table for Roles & Users
- Prints AST Tree

5. Symbol Table Construction & Semantic Analysis

Purpose

The symbol table is a central data structure that stores information about **roles, users, permissions, and inheritance** defined in the DSL. Semantic analysis ensures the DSL program is meaningful and consistent, detecting errors that the parser cannot catch.

Symbol Table Structure

- **Roles:** Each role stores:
 - Name
 - Permissions
 - Parent roles (for inheritance)
- **Users:** Each user stores:
 - Name
 - Assigned role

Semantic Analysis Tasks

1. **Undefined References:** Checks if a role inherits from another role that does not exist.
2. **Roles Without Permissions:** Checks if a role has no permissions assigned.